

Ordering System API

Relatorio tecnico detalhado do projeto NestJS, GraphQL, PostgreSQL, Redis, Kafka, Prisma e Docker

Versao do projeto: 1.0.0

Gerado em: 01/06/2026

Escopo do documento

Este PDF explica o objetivo do projeto, a stack, a arquitetura, o modelo de dados, os fluxos principais, o papel de cada tecnologia e o que cada modulo/codigo implementado faz.

Indice

- 1. Visao Geral Do Projeto
 - 2. Arquitetura E Estrutura De Pastas
 - 3. Modelo De Dados Com Prisma E PostgreSQL
 - 4. O Que Cada Parte DoCodigo Faz
 - 5. Fluxo Detalhado De Criacao De Pedido
 - 6. Redis E Kafka
 - 7. Docker, Ambiente E Execucao
 - 8. API GraphQL
 - 9. Boas Praticas Aplicadas
 - 10. Proximos Passos Recomendados

Documentacao Tecnica

1. Visao Geral Do Projeto

O Ordering System API e um backend profissional para gerenciamento de pedidos, construido em Node.js com NestJS e GraphQL. O projeto foi desenhado para portfolio, mas usa decisoes reais de arquitetura: separacao por modulos, services com regra de negocio, resolvers finos, Prisma como camada de persistencia, Redis para cache e Kafka para processamento assincrono.

A API permite criar usuarios, cadastrar produtos, criar pedidos, listar pedidos, buscar pedido por ID e atualizar status. Ao criar um pedido, o sistema salva os dados no PostgreSQL, invalida cache no Redis, publica um evento no Kafka e um consumer processa esse evento para simular a evolucao assincrona do pedido.

Objetivo tecnico

Demonstrar uma arquitetura backend moderna com GraphQL, persistencia relacional, cache distribuido, mensageria, containers, validacao de entrada e configuracao por variaveis de ambiente.

Stack implementada

| Tecnologia | Papel no projeto |
|----------------|--|
| NestJS | Framework principal da API. Organiza modulos, injecao de dependencia, bootstrap, providers e ciclo de vida da aplicacao. |
| GraphQL | Camada de contrato da API. Expõe queries e mutations como createUser, findProducts, createOrder e updateOrderStatus. |
| PostgreSQL | Banco relacional responsavel por armazenar usuarios, produtos, pedidos e itens de pedido. |
| Prisma ORM | Mapeia o schema relacional para TypeScript e fornece queries tipadas, migrations e transacoes. |
| Redis | Cache de leituras frequentes, principalmente listagem de produtos e busca de pedido por ID. |
| Apache Kafka | Mensageria para desacoplar a criacao do pedido do processamento assincrono. |
| Docker Compose | Orquestra app, postgres, redis, kafka e zookeeper em ambiente local reproduzivel. |

| Tecnologia | Papel no projeto |
|----------------------|---|
| class-validator | Valida inputs GraphQL antes de executar a regra de negocio. |
| @nestjs/config + Joi | Carrega e valida variaveis de ambiente obrigatorias. |
| TypeScript | Tipagem estatica, modelos GraphQL, DTOs e maior seguranca de refatoracao. |

2. Arquitetura E Estrutura De Pastas

A arquitetura segue uma abordagem modular. Cada area de negocio possui seu proprio module resolver, service, DTOs de entrada e models de saida. Integracoes de infraestrutura tambem sao isoladas em modulos dedicados, como Redis, Kafka e Prisma.

```
src/  
  main.ts  
  app.module.ts  
  config/  
  prisma/  
  common/  
    exceptions/  
    filters/  
    interceptors/  
  modules/  
    users/  
    products/  
    orders/  
    redis/  
    kafka/  
  prisma/  
    schema.prisma  
    migrations/  
  docker-compose.yml  
  Dockerfile  
  .env  
  .env.example  
  README.md
```

Fluxo entre camadas

1. O cliente envia uma query ou mutation para /graphql.
2. O resolver GraphQL recebe os argumentos e chama um service.
3. O service executa regra de negocio, validacoes de existencia e operacoes

4. O PrismaService executa queries no PostgreSQL.
5. O RedisService armazena ou invalida cache quando necessario.
6. O KafkaProducerService publica eventos de dominio.
7. O KafkaConsumerService processa eventos de forma assincrona.

Principios usados

- Resolvers nao contem regra de negocio; eles apenas adaptam GraphQL para services.
- Services concentram regras como calculo de total, validacao de estoque e atualizacao de status.
- DTOs/InputTypes representam entrada e usam class-validator.
- Models/ObjectTypes representam retorno GraphQL.
- Infraestrutura externa fica encapsulada para reduzir acoplamento.

3. Modelo De Dados Com Prisma E PostgreSQL

O arquivo prisma/schema.prisma define o contrato entre a aplicacao e o banco de dados. As entidades principais: User, Product, Order e OrderItem, alem do enum OrderStatus. O modelo e usado por ser relacional, consistente e adequado para pedidos, itens e inventario.

| | Entidade | Campos principais | Responsabilidade |
|-----------|---|--|------------------|
| User | id, name, email, createdAt, updatedAt | Representa a pessoa que realiza pedidos. O email e unico. | |
| Product | id, name, description, price, stock, createdAt, updatedAt | Representa produtos disponiveis para compra. Price usa Decimal no banco para evitar erro de ponto flutuante. | |
| Order | id, userId, status, total, createdAt, updatedAt | Representa o pedido. Possui usuario, status e total consolidado. | |
| OrderItem | id, orderId, productId, quantity, price | Representa os itens do pedido. Guarda o preco historico do produto no momento da compra. | |

Relacionamentos

- User possui muitos Orders.
- Order pertence a um User.
- Order possui muitos OrderItems.
- OrderItem pertence a um Order e a um Product.
- Product pode aparecer em muitos OrderItems.

Status do pedido

| Status | Significado |
|------------|--|
| PENDING | Pedido criado e aguardando processamento. |
| PROCESSING | Consumer Kafka iniciou processamento assíncrono. |
| APPROVED | Pedido processado e aprovado. |
| CANCELED | Pedido cancelado manualmente por mutation. |

Migration inicial

A migration `prisma/migrations/20260601203800_init/migration.sql` cria o enum `OrderStatus`, as tabelas, índices e foreign keys. Isso permite que o ambiente Docker aplique o schema automaticamente com `prisma migrate deploy` ao iniciar o container app.

4. O Que Cada Parte Do Código Faz

`src/main.ts`

É o ponto de entrada da aplicação. Cria a instância Nest, registra `ValidationPipe` global, configura `GraphQL` de execuções, interceptor de logging, habilita `CORS` e inicia o servidor na porta configurada.

- `ValidationPipe` com whitelist remove propriedades desconhecidas.
 - `forbidNonWhitelisted` rejeita campos não declarados nos DTOs.
 - `transform` permite converter entradas para tipos esperados.

`src/app.module.ts`

É o módulo raiz. Carrega `ConfigModule` global, configura `GraphQL` com Apollo, registra `autoSchemaFile`, registra `PrismaModule`, `RedisModule`, `UsersModule`, `ProductsModule`, `OrdersModule`. Como o `KafkaModule` é usado dentro de `OrdersModule`, ele fica disponível para pedidos.

`src/config`

`configuration.ts` organiza variáveis de ambiente em grupos lógicos: app, database, kafka. `env.validation.ts` usa Joi para garantir que variáveis obrigatórias existam e tenham o tipo correto antes da aplicação iniciar.

`src/prisma`

`PrismaModule` é global e exporta `PrismaService`. `PrismaService` estende `PrismaClient` e implementa `onModuleInit/onModuleDestroy` para abrir e fechar conexão com o banco.

de vida do Nest.

src/modules/users

| | Arquivo | Funcao |
|----------------------|---|--------|
| create-user.input.ts | Define name e email de entrada. Usa IsString, IsNotEmpty, MaxLength e IsEmail. | |
| user.model.ts | Define o tipo GraphQL de saida UserModel. | |
| users.resolver.ts | Expoe createUser e findUsers no schema GraphQL. | |
| users.service.ts | Cria usuarios, lista usuarios e trata conflito de email unico com Prisma P2002. | |
| users.module.ts | Agrupar resolver e service e exporta UsersService. | |

src/modules/products

| | Arquivo | Funcao |
|-------------------------|--|--------|
| create-product.input.ts | Define name, description, price e stock com validacoes. | |
| product.model.ts | Define ProductModel para GraphQL, incluindo price como Float e stock como Int. | |
| products.resolver.ts | Expoe createProduct e findProducts. | |
| products.service.ts | Cria produtos, converte price para Prisma.Decimal, lista produtos e aplica cache Redis em products:list. | |
| products.module.ts | Agrupar resolver/service e importa RedisModule. | |

src/modules/orders

| | Arquivo | Funcao |
|------------------------------|---|--------|
| create-order.input.ts | Recebe userId e uma lista de itens. Usa ArrayNotEmpty e ValidateNested. | |
| order-item.input.ts | Recebe productId e quantity, garantindo quantidade minima 1. | |
| update-order-status.input.ts | Recebe orderId e status validado contra OrderStatus. | |
| order-status.enum.ts | Define enum local usado por GraphQL e class-validator. Evita dependencia de enum runtime do Prisma. | |

| Arquivo | Funcao |
|---------------------|---|
| order.model.ts | Define retorno GraphQL do pedido, incluindo user e items. |
| order-item.model.ts | Define retorno GraphQL dos itens do pedido e produto opcional. |
| orders.resolver.ts | Expor createOrder, findOrders, findOrderById e updateOrderStatus. |
| orders.service.ts | Implementa regra de negocio: valida usuario/produtos, calcula total, cria pedido em transacao, decrementa estoque, invalida cache e publica evento Kafka. |

src/modules/redis

RedisService encapsula ioredis. Ele fornece get, set, del e delByPattern. Isso evita que services conhecam detalhes do cliente Redis e padroniza TTL e serializacao JSON.

src/modules/kafka

KafkaProducerService cria o producer e publica order.created. KafkaConsumerService cria o consumer, assina o topico order.created e atualiza o pedido para PROCESSING e depois para APPROVED. Ambos usam conexao resiliente para evitar que o app caia se o Kafka ficar

src/common

- GraphQLExceptionFilter permite que HttpException seja retornada de forma correta no GraphQL.
- LoggingInterceptor mede tempo de execucao de handlers e ajuda a observar o desempenho.
- AppException e uma excecao base simples para padronizar erros customizados.

5. Fluxo Detalhado De Criacao De Pedido

A criacao de pedido e o principal caso de uso do sistema. Ela combina transacao SQL, Redis e Kafka.

1. Cliente chama createOrder com userId e items.
2. CreateOrderInput valida formato, lista nao vazia e itens aninhados.
3. OrdersResolver delega para OrdersService.create.
4. O service agrupa itens duplicados do mesmo produto.
5. Uma transacao Prisma valida usuario, busca produtos e calcula total.
6. O total e calculado somando product.price * quantity.
7. O Order e criado com status PENDING.

8. O estoque é decrementado com `updateMany` e condição `stock >= quantity`.
9. O cache `orders:*` é invalidado para evitar leituras antigas.
10. `KafkaProducerService` publica `order.created`.
11. `KafkaConsumerService` consome o evento, muda status para `PROCESSING`, aguarda simulação e muda para `APPROVED`.
12. A cada atualização assíncrona, o cache `orders:{id}` é invalidado.

Por que usar transação?

A transação garante que a criação do pedido, itens e atualização de estoque aconteçam como uma unidade lógica. Se alguma parte falhar, tudo é revertido. Isso evita pedido criado sem item, decrementado sem pedido ou total incoerente.

Por que salvar preço no `OrderItem`?

O preço do produto pode mudar depois. Guardar preço dentro de `OrderItem` preserva o valor histórico pago naquele pedido, uma prática importante em sistemas de venda reais.

6. Redis E Kafka

Cache com Redis

Redis é usado para reduzir consultas repetidas ao banco em leituras frequentes. O projeto usa cache em `findProducts` e `findOrderById`.

| | Operação | Chave | Quando invalida |
|----------------------------|----------------------------|--|-----------------|
| <code>findProducts</code> | <code>products:list</code> | Quando <code>createProduct</code> cria um novo produto. | |
| <code>findOrderById</code> | <code>orders:{id}</code> | Quando pedido é criado, status é atualizado ou consumer processa evento. | |

A invalidação é explícita. Isso evita retornar produtos desatualizados depois de alterações.

Mensageria com Kafka

Kafka representa uma fila/event stream entre a criação do pedido e a etapa assíncrona. Ela salva o pedido e publica o evento. O `KafkaConsumerService` precisa processar tudo sincronamente. Ela salva o pedido e publica o evento. O `KafkaProducerService` cuida da etapa assíncrona.

| | Componente | Responsabilidade |
|-----------------------------------|--|------------------|
| <code>KafkaProducerService</code> | Conectar ao Kafka e publicar <code>order.created</code> com <code>orderId</code> , <code>userId</code> , <code>total</code> e <code>createdAt</code> . | |

| Componente | Responsabilidade |
|----------------------|---|
| KafkaConsumerService | Assinar order.created, simular processamento e atualizar status no banco. |
| OrderCreatedEvent | Contrato do payload publicado no topico. |
| Zookeeper | Coordena o broker Kafka na imagem usada pelo docker-compose. |

7. Docker, Ambiente E Execucao

O Dockerfile usa multi-stage build. Primeiro instala dependencias, depois gera Prisma Client, compila NestJS, e no estagio final copia dist, prisma e node_modules. O Prisma Client é gerado no estagio final para garantir que o runtime tenha o client inicializado.

Servicos do docker-compose

| | | Servico | Imagem/Build | Porta | Funcao |
|-----------|---------------------------------|---------|--------------|-----------------------|--------|
| app | Dockerfile local | | 3000 | API NestJS GraphQL. | |
| postgres | postgres:16-alpine | | 5432 | Banco relacional. | |
| redis | redis:7-alpine | | 6379 | Cache. | |
| zookeeper | confluentinc/cp-zookeeper:7.7.0 | | 2181 | Coordenacao do Kafka. | |
| kafka | confluentinc/cp-kafka:7.7.0 | | 9092/29092 | Broker de eventos. | |

Healthchecks e prontidao

PostgreSQL possui healthcheck com pg_isready. O app depende de postgres healthy e kafka versions. O app depende de postgres healthy e kafka versions.

Comandos principais

```
npm install
npm run prisma:generate
npm run build
npm run lint
npm run docs:pdf
docker compose up --build
docker compose down
```

8. API GraphQL

O projeto usa GraphQL code-first. Os decorators @ObjectType, @InputType, @Field, @Query e @Mutation geram o schema GraphQL a partir do TypeScript.

Operacoes disponiveis

| Operacao | Tipo | Descricao |
|-------------------|----------|--|
| createUser | Mutation | Cria usuario com name e email. |
| findUsers | Query | Lista usuarios cadastrados. |
| createProduct | Mutation | Cria produto com preco e estoque. |
| findProducts | Query | Lista produtos usando cache Redis. |
| createOrder | Mutation | Cria pedido, calcula total, decrementa estoque e publica evento. |
| findOrders | Query | Lista pedidos com usuario, itens e produtos. |
| findOrderById | Query | Busca pedido por ID usando cache Redis. |
| updateOrderStatus | Mutation | Atualiza manualmente o status do pedido. |

Exemplo: criar usuario

```
mutation CreateUser {
  createUser(input: {
    name: "Ana Silva"
    email: "ana@example.com"
  }) {
    id
    name
    email
  }
}
```

Exemplo: criar produto

```

mutation CreateProduct {
  createProduct(input: {
    name: "Mechanical Keyboard"
    description: "RGB keyboard with blue switches"
    price: 299.90
    stock: 10
  }) {
    id
    name
    price
    stock
  }
}

```

Exemplo: criar pedido

```

mutation CreateOrder {
  createOrder(input: {
    userId: "USER_ID"
    items: [
      { productId: "PRODUCT_ID", quantity: 2 }
    ]
  }) {
    id
    status
    total
    items {
      quantity
      price
      product { name }
    }
  }
}

```

9. Boas Praticas Aplicadas

- Arquitetura modular por dominio e infraestrutura.
 - Separacao entre entrada GraphQL, saida GraphQL e regra de negocio.
 - Uso de transacao para operacoes criticas de pedido e estoque.
 - Uso de Decimal no Prisma para valores monetarios.
 - Cache com invalidacao explicita.
 - Mensageria para desacoplar processamento do pedido.

- Docker Compose reproduzível para stack completa.
 - Configuração por variáveis de ambiente com validação.
 - Build, lint e Prisma validate executados durante o desenvolvimento.

Decisões tomadas durante a estabilização

| Problema encontrado | Ajuste aplicado |
|--|---|
| PowerShell bloqueava npm.ps1 | Uso de npm.cmd nos comandos Windows. |
| Prisma validate sem DATABASE_URL | Criação de .env local e Compose com hosts internos fixos. |
| Prisma Client não inicializado no container | Dockerfile passou a executar npx prisma generate também no estágio final. |
| Kafka ainda não pronto no boot | Healthcheck no Kafka e conexão resiliente em producer/consumer. |
| Enum Prisma indefinido em runtime para class-validator | Enum OrderStatus local para GraphQL e validação. |
| GraphQL não inferia description nullable | Tipo GraphQL explícito e normalização de null para undefined. |
| Apollo/Nest exigia integração Express 5 | Adição de @as-integrations/express5. |

10. Próximos Passos Recomendados

O projeto já demonstra uma base profissional, mas pode evoluir para um sistema completo e próximo de produção.

- Adicionar autenticação JWT e autorização por roles.
 - Criar testes unitários para services e testes e2e para GraphQL.
 - Adicionar paginação e filtros nas listagens.
 - Implementar updateProduct e regras de reposição de estoque.
 - Adicionar observabilidade com logs estruturados, metrics e traces.
 - Criar DLQ ou tópico de erro para falhas no consumer Kafka.
 - Adicionar CI com lint, build, testes e prisma validate.
 - Criar seed de dados para demonstração do produto.
 - Adicionar tratamento de idempotência para criação de pedidos.

Resumo executivo

Resultado

O projeto ordering-system entrega uma API GraphQL modular com persistencia relacional, cache, mensageria e containerizacao. Ele e adequado para portfolio porque mostra dominio de arquitetura backend moderna, integracoes reais e preocupacao com operacao local reproduzivel.

